CPCS-403 | Homework 1 Report

ESP32 Guide Document

Group members:
1- Tariq Mohammed Areesh
2- Majd Ahmed Al-farasani
IDs:
1- 2237498
2- 2237426
Section: CS1
Date: 2025/12/10

ESP32 Guide Website Link:

https://esp32hub.infinityfree.me/pages/index.php

1. Idea Development

This final project continues and expands the idea from Homework 1 - ESP32 Guide. In Homework 1, the website was a simple, static guide designed to help beginners understand what the ESP32 microcontroller is, how to set it up, and how to use it in basic projects. It focused mainly on explaining concepts and providing step-by-step learning resources for new students.

In this final project, the idea grows into a fully-featured ESP32 Hub web application. Instead of just being a static guide, it becomes an interactive platform where users can:

- Create accounts and log in.
- View, add, and manage ESP32 learning projects.
- Browse a gallery of ESP32-related images.
- Watch videos related to ESP32 tutorials.
- Upload help requests and view them later.
- Submit feedback through a validated form.
- Send contact messages.
- Access a resume page.

The main goal of the ESP32 Hub is to be a centralized platform for ESP32 learners and hobbyists, combining learning materials, user interaction, and project/issue management in one web application. It keeps the beginner-friendly spirit of the original ESP32 Guide, but adds database integration, backend logic, security, and responsive design as required in the final project description.

2. Website Functions

The web application is organized into multiple PHP pages. Each page has a specific purpose and is connected to the database and backend logic through includes such as db_connect.php, header.php, footer.php, and auth_check.php.

Below is an overview of the main pages:

1- Home (index.php)

- Introduces the ESP32 Hub platform.
- Explains the purpose of the website and highlights key sections such as Projects, Gallery, Videos, Help, and Feedback.

- Displays the main navigation menu, which is shared across all pages using a common header include.

2- Register (register.php)

- Allows new users to create an account.
- Collects basic information such as username, email, and password.
- Validates inputs and stores the data in the users table.
- Prevents duplicate usernames or emails.

3- Login (login.php)

- Authenticates existing users using their email/username and password.
- Starts a session for the user and redirects to the Dashboard on success.
- Displays error messages for invalid credentials.

4- Logout (logout.php)

- Ends the current user session.
- Redirects back to the Home or Login page.
- Ensures that session data is properly cleared.

5- Dashboard (dashboard.php)

- A protected page that only authenticated users can access.
- Welcomes the logged-in user by name.
- Provides quick links to manage projects, view help requests, open the feedback form, and access resources.

6- Projects (projects.php)

- Displays a list of ESP32 learning projects stored in the projects table.
- Each project shows:
    - Category (e.g., Getting Started, Sensors, IoT).
    - Title.
    - Difficulty (Beginner / Intermediate / Advanced).
    - Estimated time.
    - Short description.
- Initial example data is inserted from the SQL script so that the page is already populated with sample projects.

7- Add Project (add_project.php)

- Contains a form that allows logged-in users to add a new ESP32 learning project.
- Provides dropdowns and text fields to select the project category, difficulty, estimated time, and description.
- Sends the data to the server, where it is validated and inserted into the projects table with a proper foreign key to categories.
- Confirms successful insertion or shows error messages.

8- Delete Project (delete_project.php)

- Handles deletion of projects from the projects table.
- Uses the project ID to identify which project to delete.
- Ensures that only authorized, logged-in users can perform this action.

9- Gallery (gallery.php)

- Implements the image gallery requirement from the project description.
- Displays a set of ESP32-related images as thumbnails in a grid.
- Uses JavaScript (gallery.js) so that when the user clicks on a thumbnail, the larger version of the image is shown in a main preview area.
- Improves visual learning by showcasing ESP32 boards, wiring setups, and project photos.

10- Videos (videos.php)

- Implements the video requirement.
- Embeds one or more tutorial videos about ESP32, either from local files in the /videos folder or from YouTube.
- Uses a clean layout (defined in style.css) to present the video player and accompanying descriptions.
- Allows users to watch learning content directly inside the website.

11- Resources / Table Page (resources.php)

- Contains a dynamic table generated from database data (projects or other tabular content) displayed using a <table> element styled in style.css.
- The table is filled using PHP and MySQL and can make use of colspan/rowspan if needed.

- This page also supports printing a clean version of the table using a separate print stylesheet (print.css), which hides navigation and buttons when printing.

12- Resume (resume.php)

- Displays the PDF version of the resume within the page using the <object> tag, as required by the project description.
- Allows visitors or instructors to view the resume directly without downloading it or opening a new tab.

13- Help Upload (help_upload.php)

- Provides a form where users can submit help requests about their ESP32 problems.
- Lets users describe the issue and optionally upload a related file (e.g., screenshot or code).
- Saves help data into the help_requests table, linked to the logged-in user.

14- Help List (help_list.php)

- Shows a list of all help requests stored in the system.
- For each request, displays title, user, and submission date.
- Each item includes a link to view more details.

15- Help View (help_view.php)

- Shows the complete details of a selected help request:
    - Title
    - Description
    - File path (if any)
    - Date submitted
- Helps instructors or advanced users understand the problem and provide guidance.

16- Feedback (feedback.php)

- Implements the main feedback form required by the project.
- Contains multiple fields:
    - Name
    - Email
    - Project title

    - Experience level
    - Feedback type (radio buttons)
    - Improvement areas (checkboxes)
    - Source (dropdown: how user heard about ESP32 Hub)
    - Detailed message (textarea)
  - Uses JavaScript and PHP validation before storing data into the feedback table.

17- Contact (contact.php)

  - Provides contact information for the website owner (name, email, possibly GitHub).
  - Can also include a simple contact form whose submissions are stored in the contact_messages table.

All pages use a common header and footer with PHP includes, and the layout is styled using an external CSS file (style.css).

3. Recommended Solution

Students and beginners who want to learn ESP32 often struggle because information is:

- Scattered across many websites.
- Not interactive.
- Not connected to real projects or a community.

In Homework 1, the ESP32 Guide partially solved this by giving one place for clear explanations, pinout references, installation steps, and example projects.

The final project takes the next step by turning the idea into a dynamic ESP32 Hub with:

- User accounts and authentication so users can have a personalized experience.
- Project management so they can browse suggested ESP32 projects by category, difficulty, and time.
- Help requests and help solutions, allowing learners to ask for assistance when they face problems.
- Feedback forms and contact pages to support continuous improvement and communication between learners and the site owner.
- Media content (gallery and videos) to make learning more visual and engaging.
- Database-driven content to keep data organized, searchable, and scalable.

This solution matches the final project requirements of having multiple PHP pages, forms, validation, database integration, security features, and responsive design.

4. Benefits

The ESP32 Hub provides several benefits for students and beginners:

4.1 Centralized ESP32 platform

All projects, help requests, videos, images, and feedback are collected in one web application. Users no longer have to jump between many different resources.

4.2 Interactive learning

Instead of just reading, users can:

- Log in to their own account.
- Explore structured project ideas.
- Submit feedback and help requests.
- Interact with tables, forms, gallery, and videos.

4.3 Real project management

Students can see structured project descriptions with categories, difficulty, and estimated time. This connects theoretical ESP32 knowledge with practical project ideas.

4.4 Practical exposure to web technologies

Developing and using this site combines:

- HTML and CSS for structure and design.
- JavaScript for interactivity and validation.
- PHP for backend processing.
- MySQL for database storage.

This helps the developer gain full-stack web development skills.

4.5 Time saving

Information, videos, images, and project ideas are in one place, so users do not need to search the internet for every tutorial.

4.6 Support and feedback

Help Upload and Feedback pages allow two-way communication. Users can ask questions and submit detailed feedback, which improves the content and user experience.

4.7 Professional touch

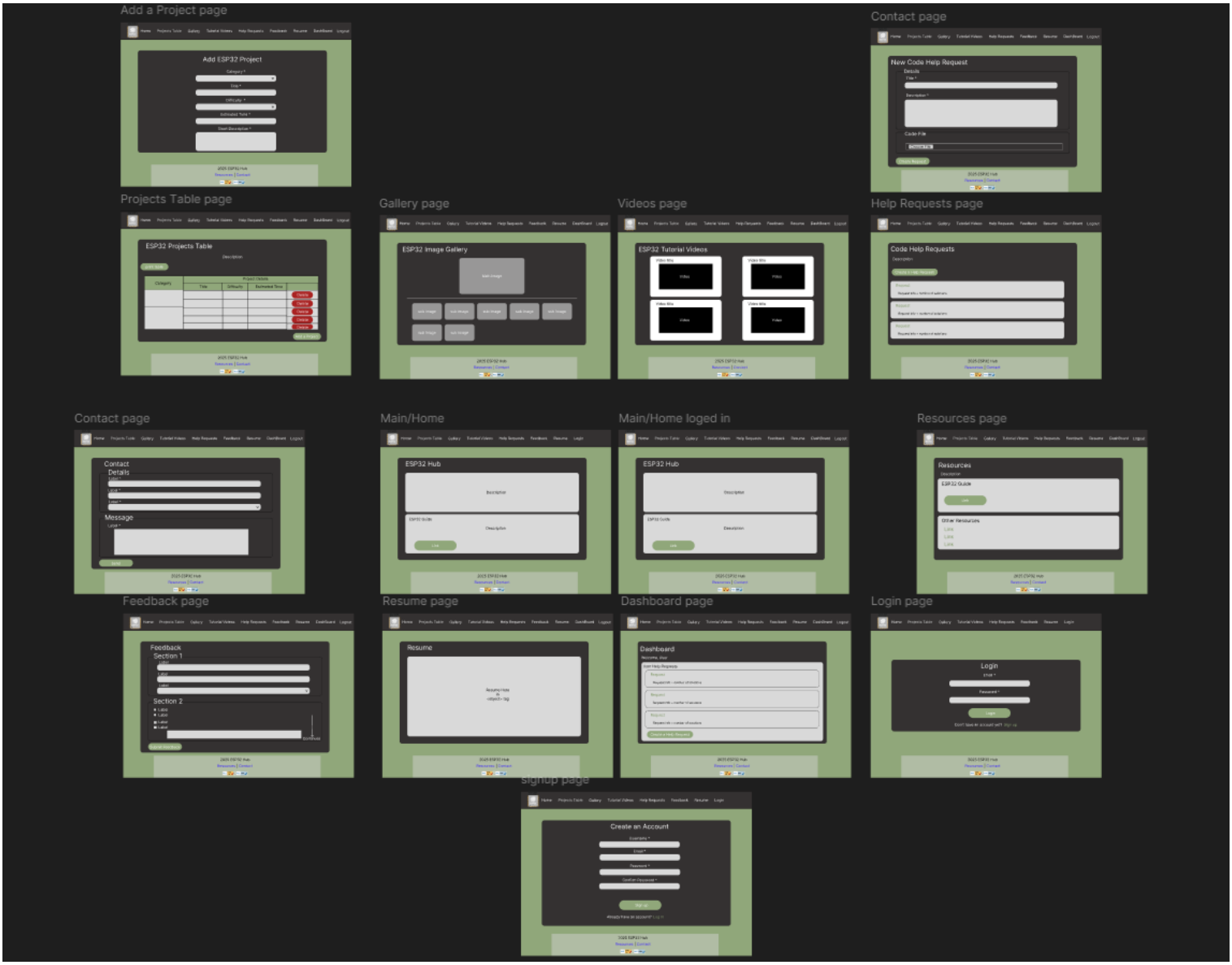
The Resume and Contact pages make the site suitable to share with instructors or future employers as a portfolio, demonstrating both ESP32 knowledge and web development skills.

5. Storyboard

Here is a link to the figma prototype for clear visualization of the Storyboard:

Figma link: https://www.figma.com/design/2gipZQA7SZin0AIYoB1Sln/Esp32-Hub?node-id=0-1&p=f&t=DGDzq9MBq25VEEBC-0

Prototype link: https://www.figma.com/proto/2gipZQA7SZin0AIYoB1Sln/Esp32-Hub?node-id=0-1&t=DGDzq9MBq25VEEBC-1



6. Database Design

The ESP32 Hub uses a MySQL database hosted with the website, as required in the final project description.

The actual SQL script defines the following main tables:

6.1 users

- user_id (INT, primary key, auto increment)
- username (VARCHAR, unique)
- email (VARCHAR, unique)
- password_hash (VARCHAR)
- created_at (TIMESTAMP, default current time)

This table stores login information for all users.

6.2 categories

- category_id (INT, primary key, auto increment)
- category_name (VARCHAR)

Stores project categories such as “Getting Started”, “Sensors & Inputs”, “Outputs & Actuators”, and “IoT & Connectivity”. These are inserted by default in the SQL script.

6.3 projects

- project_id (INT, primary key, auto increment)
- category_id (INT, foreign key → categories.category_id)
- title (VARCHAR)
- difficulty (VARCHAR)
- estimated_time (VARCHAR)
- short_description (TEXT)

Represents example ESP32 learning projects. Each project belongs to exactly one category.

6.4 feedback

- feedback_id (INT, primary key, auto increment)
- name (VARCHAR)
- email (VARCHAR, unique)
- project_title (VARCHAR)
- experience_level (VARCHAR)
- feedback_type (VARCHAR)
- improvements (VARCHAR - stores selected checkboxes as a string)
- source (VARCHAR - how the user heard about ESP32 Hub)
- message (TEXT)
- created_at (TIMESTAMP)

This table stores structured feedback from users. The unique email constraint ensures that each email address can submit feedback only once, matching the spec.

6.5 contact_messages

- message_id (INT, primary key, auto increment)
- name (VARCHAR)
- email (VARCHAR)
- subject (VARCHAR)
- message (TEXT)
- created_at (TIMESTAMP)

Stores messages sent from the Contact page.

6.6 help_requests

- request_id (INT, primary key, auto increment)
- user_id (INT, foreign key → users.user_id)
- title (VARCHAR)
- description (TEXT)
- file_path (VARCHAR, optional)
- created_at (TIMESTAMP)

Represents help tickets submitted by users.

6.7 help_solutions

- solution_id (INT, primary key, auto increment)
- request_id (INT, foreign key → help_requests.request_id)
- user_id (INT, foreign key → users.user_id)
- solution_text (TEXT)
- created_at (TIMESTAMP)

Allows saving solutions or responses to help requests.

6.8 Relationships

- One category can have many projects (one-to-many).
- One user can submit many help_requests and help_solutions (one-to-many).
- feedback uses email as a natural key to prevent duplicates.

The PHP pages use this database to:

- Insert new users, projects, feedback, help requests, help solutions, and contact messages.
- Display project lists, resource tables, and help lists dynamically based on database contents.

7. Forms, Validation, and Feedback

The project includes at least one main form that satisfies all the detailed requirements from the final project brief.

7.1 Feedback Form (feedback.php)

Form design

- Uses <fieldset> and <legend> to separate sections such as User Information and Feedback Details.
- Contains the following elements:
    - 4 textboxes:
        - Name
        - Email
        - Project title
        - Experience level (or similar text entry)
    - Radio buttons (feedback type, e.g. “General feedback”, “Bug report”).
    - Checkboxes (improvement options, e.g. “More projects”, “Better design”, “More videos”).
    - Dropdown list (source of discovery: “Friend”, “Search engine”, “University course”, etc.).
    - Textarea (detailed feedback message).
- All fields are styled using the global CSS file to match the overall design.

Client-side validation (JavaScript)

- Implemented in validation.js.

validation

- Ensures that:
    - Name is not empty.
    - Email is not empty and matches a valid email pattern.
    - Project title and experience level are provided.
    - Feedback type (radio) is selected.
    - At least one improvement checkbox is selected.
    - Source (dropdown) is chosen.

    - Message textarea is not empty.
- If validation fails, the script:
    - Prevents the form from submitting.
    - Displays error messages inside a designated error box, or in an alert if the box is not available.

Server-side validation (PHP)

- Re-validates all required fields to avoid bypassing JavaScript.
- Checks whether the email address already exists in the feedback table:
    - If the email is new, a new record is inserted.
    - If the email already exists, an error message is shown and the data is not inserted.

7.2 Other Forms

- Register form:
    - Validates that username, email, and password are provided.
    - Ensures that email is unique and password meets minimum length.
- Login form:
    - Ensures email/username and password are entered.
    - Uses password_verify() to compare the password with the stored hash.
- Add Project form:
    - Checks that category, title, difficulty, estimated time, and description are provided.
- Help Upload form:
    - Validates title and description.
    - If a file is uploaded, checks its type and size as needed.

These forms collectively satisfy the form, validation, and server-processing requirements of the final project.

8. Security and Authentication

To meet the security part of the project requirements, the ESP32 Hub addresses the following points:

8.1 Client-side and Server-side Validation

All important input fields are validated in JavaScript and then again in PHP. This ensures that even if a user disables JavaScript, invalid or malicious data cannot easily reach the database.

8.2 SQL Injection Protection

- Database queries are designed to use parameter binding / prepared statements (e.g. mysqli with bound parameters).
- User inputs are never directly concatenated into SQL strings without sanitization.

8.3 Cross-Site Scripting (XSS) Protection

- When displaying data submitted by users (such as project titles, help descriptions, or feedback messages), output is escaped using functions like htmlspecialchars() so that any HTML tags are treated as text instead of code.

8.4 Authentication and Authorization

- Passwords are stored as secure hashes using password_hash().
- During login, password_verify() is used to check passwords.
- After login, a session variable such as $_SESSION['user_id'] is set.
- Protected pages (dashboard.php, add_project.php, help_upload.php, etc.) include an auth_check.php file which:
    - Verifies that the session is active.
    - Redirects to the Login page if the user is not authenticated.

8.5 Secure Design Principles

- Only logged-in users can add or delete projects or submit help requests.
- Direct URLs to protected pages cannot be accessed without a valid session.
- Error messages are written to be informative but not to reveal internal implementation details such as SQL queries or file paths.

Together, these measures ensure that the application is reasonably secure for a student project level.

9. Responsive Design and Hosting

The final project requires a responsive website that looks good on different devices (desktop, tablet, and mobile).

9.1 Responsive Design

The website uses a main external CSS file (style.css) that defines layout using flexbox and CSS grid.

- The main components (header, footer, content area, cards, and tables) are arranged using containers and modern layout techniques.
- Multiple media queries are defined:
    - For desktop (≥ 1024px), the container width is increased, and grids show more columns.
    - For tablet (768px-1023px), padding and grid columns are adjusted to fit.
    - For smartphones (≤ 767px), the navigation becomes compact, cards stack vertically, tables allow horizontal scrolling, and auth forms become full-width.

This ensures that:

- Navigation remains usable on small screens.
- Content is readable without horizontal scrolling where possible.
- Certain elements (like galleries, grids, and video layouts) automatically adapt to a single column on mobile.

9.2 Print Stylesheet

The project includes print.css, a separate CSS file used for printing the dynamic table:

print.css

- Hides all non-table elements (navigation, buttons, etc.) by setting visibility: hidden on most elements and visibility: visible for #print-area.
- Ensures the printed page only shows the table contents and caption.
- Hides action buttons such as “Print” or “Delete” during printing.

9.3 Hosting and Folder Structure

The project’s root folder is organized as follows (based on your directory screenshot):

- /css - contains style.css and print.css.
- /includes - contains reusable PHP includes such as header.php, footer.php, db_connect.php, auth_check.php.

- /pages - contains the main content pages (if separated) or some may be in the root as well.
- /script - contains JavaScript files (gallery.js, validation.js, main.js).
- /images - contains gallery and UI images.
- /videos - contains local tutorial videos if used.
- /uploads -stores files uploaded with help requests.
- esp32_hub_db.sql - SQL script to create and populate the database.

The full project is uploaded to a hosting service that supports PHP and MySQL, and it is accessible via a public URL, fulfilling the requirement that the application be live online.

10. Conclusion

The ESP32 Hub final project successfully extends the original ESP32 Guide idea from Homework 1 into a more advanced, fully dynamic web application.

It now includes:

- Multiple PHP pages with a unified design and common header/footer.
- User registration, login, and protected content through sessions.
- Dynamic database-driven content such as projects, categories, help requests, solutions, and feedback.
- A validated feedback form that stores data in MySQL with client- and server-side validation.
- An image gallery, video page, resume page, and contact page that enhance learning and presentation.
- Security measures against SQL injection and XSS, and proper authentication checks.
- Responsive design that works on desktop, tablet, and mobile devices, plus a printable table layout for resources.

Overall, this project demonstrates how a simple idea (an ESP32 learning guide) can be transformed into a complete web application that supports learning, interaction, and project management for ESP32 students and beginners, while also showcasing solid web development skills in HTML, CSS, JavaScript, PHP, and MySQL.